# Feature Engineering — INARA Project

INARA: Investigating Non-trivial Atmospheres for Retrieval Algorithms

## What Is This Project?

INARA is an exoplanet atmospheric retrieval ML project. The goal: given a simulated atmospheric climate profile, **predict the surface concentrations of 12 molecules** in an exoplanet's atmosphere.

## Raw Data

Each simulation sample comes from a `.tar.gz` archive containing:

| File | Shape | Content |
|---|---|---|
| `parsed_clima_final.npy.npz` | (101, 12) | CLIMA atmospheric profile |
| `parsed_photochem_mixing_ratios.npy.npz` | (102, 66) | Photochemical mixing ratios |
| `mixing_ratios.dat` | text | Surface species concentrations (for N2) |
| `run_metadata.json` | scalars | Fluxes, pressure, temperature |

## Step 1 — Raw Feature Extraction (process_inara.py)

### CLIMA Profile → Input Features

The CLIMA array is **(101 altitude levels × 12 channels)**. The 12 channels are:

| Index | Name | Physical Meaning |
|---|---|---|
| 0 | J | Level index (1–101) |
| 1 | P | Atmospheric pressure |
| 2 | ALT | Altitude (km) |
| 3 | T | Temperature |
| 4 | CONVEC | Convective flux |
| 5 | DT | Temperature time derivative |
| 6 | TOLD | Previous temperature (solver state) |
| 7 | FH2O | Water vapor flux |
| 8 | FSAVE | Saved flux |
| 9 | FO3 | Ozone flux |
| 10 | TCOOL | Cooling rate |
| 11 | THEAT | Heating rate |

**Transformation:** The raw array is **transposed** from `(101, 12)` → `(12, 101)` — channels-first layout, ready for 1D CNNs.

### Molecular Targets

The surface concentrations (row 0) of 11 molecules are pulled from the photochem array by column index. **N2 is special** — it's parsed from `mixing_ratios.dat`, with fallback: `N2 ≈ max(0, 1 - H₂O - O₂)`.

**All 12 molecule concentrations are $\log_{10}$-transformed** with a floor of −40 to avoid −∞ for trace species:

```
result = np.log10(np.abs(arr))
return np.clip(result, -40.0, None)
```

**Why $\log_{10}$?** Molecular concentrations span many orders of magnitude. Log-space compresses this to a learnable range.

**Output arrays saved:**

- `spectra.npy` — shape `(N, 12, 101)` — CLIMA profiles
- `molecules.npy` — shape `(N, 12)` — $\log_{10}$ targets

## Step 2 — Train/Val/Test Split + Normalization (02_feature_engineer.py)

### Split

- **70% train / 15% val / 15% test** using `sklearn.train_test_split(random_state=42)`

- Indices saved to disk so every downstream step uses the exact same split

### SpectraScaler — Per-Channel Z-Score Normalization

```
Z = (X - mean) / std
```

- Fit **only on training data**, then applied to val/test
- Statistics are **per-channel, per-altitude-level** — shape `(12, 101)` for both mean and std
- Important because different CLIMA channels live on vastly different physical scales (e.g., temperature T ≈ 200K, pressure P ≈ $10^{-6}$ to 1)
- Any std < 1e-12 is replaced with 1.0 to avoid division by zero for constant channels

### PCA Feature Extraction (for Random Forest baseline)

The Z-normalized spectra are processed in three steps:

1. **Flattened:** `(N, 12, 101)` → `(N, 1212)`
2. **PCA reduced:** `PCA(n_components=300, whiten=True, random_state=42)`
3. Output: `(N, 300)` features

**Whitening** divides each principal component by its standard deviation → all 300 features have equal variance before Random Forest training.

**300 components** capture >90% of total variance (confirmed in EDA).

## Step 3 — Two Parallel Paths After Feature Engineering

### Path A — Random Forest Baseline

- Input: `feat_{split}.npy` — 300 PCA features
- Training set capped at **10,000 samples** (intentional — fast baseline)
- **12 separate RF models**, one per molecule
- **Per-molecule hyperparameter tuning:** harder/more variable molecules (CH4, H2, CO) get deeper trees and more estimators; stable ones (N2, CO2, O2) get shallower trees

### Path B — Deep Model (SpectralResNet CNN)

- Input: `spectra_{split}.npy` — Z-normalized CLIMA profiles `(N, 12, 101)` directly — **no PCA** (the ResNet learns its own internal features)
- Uses the full training set

**Data Augmentation** (training only):

```
x = x + torch.randn_like(x) * 0.01
```

Small Gaussian noise (std=0.01) is added during training to force robust feature learning. Val/test sets are clean.

## Key Engineering Decisions

| Decision | Why |
|---|---|
| **$\log_{10}$ transform on targets** | Concentrations span 40+ orders of magnitude — log-space makes them learnable |
| **Floor at −40** | Prevents −∞ for trace molecules near zero |
| **Per-channel Z-score** (not global) | Different channels have completely different physical scales |
| **PCA whitening for RF** | Equalizes feature variance so RF splits aren't dominated by high-variance components |
| **CNN uses raw (Z-normalized) spectra** | ResNet can learn spatial patterns along altitude axis; PCA would discard structure |
| **Per-molecule RF hyperparams** | One-size-fits-all doesn't work across molecules with very different distributions |
| **Per-molecule loss weights in CNN** | Upweights rare/trace molecules (SO2, NH3 get 2× weight) to prevent training from ignoring them |
| **N2 special-cased** | Not in photochem array — requires separate parsing with two fallback strategies |

## Known Dataset Quirks

**NH3 is effectively constant** at $\log_{10} = -40$ (the floor) across all samples — std ≈ 0. The model trains a head for it but can't learn anything meaningful.

**$H_2O$ is near-constant** (std ≈ 0.00017) — very low learnability in this dataset.

## Full Pipeline Flow

```
INARA tar.gz archives
       ↓
[01_extract.py]
  - Parse CLIMA profiles → transpose (101,12) to (12,101)
  - Extract surface molecular concentrations from photochem array
  - Parse N2 from mixing_ratios.dat (with fallbacks)
  - Apply log10 transform (floor = -40) to all molecule targets
  - Save: spectra.npy (N,12,101)  |  molecules.npy (N,12)
       ↓
[02_feature_engineer.py]
```

```
  - 70/15/15 train/val/test split (random_state=42)
  - SpectraScaler: Z-score per channel (12,101) stats, fit on train only
  - Save: spectra_{train,val,test}.npy  |  molecules (unchanged log10)
  - PCA(300 components, whiten=True) on flattened normalized spectra (1212→300)
  - Save: feat_{train,val,test}.npy  |  scaler.joblib  |  pca.joblib
      ↓
   RF path                              Deep Learning path
[03_train_baseline.py]              [04_train_deep.py]
  - Input: feat (300-dim PCA)          - Input: spectra (12×101)
  - 12 per-molecule RandomForests      - SpectralResNet (1D ResNet)
  - Max 10,000 train samples           - Full dataset + Gaussian noise aug
  - Per-molecule hyperparameters       - Per-molecule loss weights
                  ↓                              ↓
          [05_evaluate.py] — R², RMSE, MAE per molecule
```